

BASTION LINE

Rapport Technique Officiel — V5

Intégration graphique Leonardo — carte + canon, sans régression gameplay

Date du rapport	28 septembre 2026
Dépôt	github.com/scuillerjack-oss/bastion-line-v0
Commit de départ (V4 vérifié)	14e8d04150661c0d484d8f98a716a242f018ce32 (référence code V4 : b9ecb8efa76dc164d4acef4614182c0e4c20e327)
Commit final V5	bb90760dac6b1d3aab043098b63edf7494276f7d
Identifiant de build vérifié	v0.1.0+bb90760 (rebuild local GITHUB_SHA=HEAD, confirmé identique)
URL publique (bêta téléphone)	https://scuillerjack-oss.github.io/bastion-line-v0/
Statut	Candidate à la bêta physique V5 — NON VALIDÉE esthétiquement, décision humaine en attente

0. Audit de l'état réel avant toute modification (cahier V5, section 2)

- Workspace / dépôt : /home/user/bastion-line-v0, dépôt Git existant scuillerjack-oss/bastion-line-v0, branche main.
- `git status` propre avant toute modification (une seule trace résiduelle non liée : `docs/generate_report_economique.py`, mission économique séparée, volontairement laissée hors de ce commit).
- HEAD local = HEAD distant (`git ls-remote`) = `14e8d04150661c0d484d8f98a716a242f018ce32`, le commit du rapport technique V4. Le commit de CODE V4 de référence exigé par le cahier, `b9ecb8efa76dc164d4acef4614182c0e4c20e327`, confirmé présent un commit plus tôt dans l'historique (jamais perdu ni réécrit).
- Version publique servie avant modification vérifiée dans la mission précédente (build ID `v0.1.0+14e8d04`, job CI "deploy" success).
- Assets fournis identifiés explicitement : une carte/terrain (1024×1024, JPEG RVB) et un canon (1024×1024, JPEG RVB) — copiés intacts dans `assets/leonardo/carte_terrain_original.jpg` et `assets/leonardo/canon_original.jpg`, jamais modifiés directement.
- Suite de tests V4 relancée AVANT toute modification : 77/77 tests unitaires passants — confirmé comme point de départ propre.

1. Intégration de la nouvelle carte Leonardo

1.1 Format et géométrie (cahier V5, section 3.1)

Mesure réelle : l'image fournie fait **1024×1024** (carrée), alors que l'espace logique du jeu (ARENA_W×ARENA_H, `engine/constants.js`) est **400×700** (ratio 0,5714) — un format portrait mobile. Un carré ne peut pas être étiré à ce ratio sans déformation visible non proportionnelle, ce qui est explicitement interdit par le cahier.

Traitement retenu (recadrage seul, jamais un redimensionnement non proportionnel) : la PLEINE hauteur de l'image est conservée (1024px) — ce qui préserve intégralement le portail haut ET la forteresse basse — et seule la largeur est recadrée à 585px (= $1024 \times 400/700$), centrée sur le chemin réellement dessiné. Le chemin mesuré oscille entre $x=57$ et $x=356$ sur les 1024px de large : entièrement contenu dans cette bande centrale de 585px, donc aucune portion du chemin, du portail ou de la forteresse n'est perdue — seul du décor latéral secondaire (rochers, arbres, un petit campement à droite) sort du cadre. Voir `scripts/process_leonardo_map.py` pour le script exact et son raisonnement documenté.

Mesure	Valeur
Dimensions source réelles	1024×1024 px (carré)
Ratio logique cible (ARENA_W/ARENA_H)	400/700 = 0,5714
Fenêtre de recadrage appliquée	$x=[219:804]$ (largeur 585), hauteur pleine 1024
Ratio obtenu	585/1024 = 0,5713 (écart < 0,02%, aucune distorsion visible)

1.2 Calage du gameplay sur la carte (cahier V5, section 3.2)

La carte fournie ne contient qu'UNE SEULE route dessinée. Son tracé a été mesuré programmatiquement (jamais à l'œil seul) : suivi du centre de la bande de couleur "route" (teinte sable, nettement distincte de l'herbe) ligne par ligne, en partant du point connu sous le portail haut et en restreignant la recherche à une

fenêtre proche de la position déjà trouvée à la ligne précédente — voir `assets/leonardo/trace_road.py`. Les points de virage obtenus (extrema locaux) ont ensuite été convertis dans l'espace logique du jeu et recopiés dans `engine/levels.js` (constante `PATH_MAP`).

Comme il n'existe qu'UN SEUL asset de carte, ce même tracé sert désormais de chemin logique pour les 4 niveaux à voie unique (1 à 4) — seules les vagues/difficultés changent d'un niveau à l'autre, jamais la géométrie du chemin, puisqu'il n'existe qu'une seule route dessinée à leur faire correspondre. Vérifié visuellement (superposition du tracé retenu sur l'image source) puis en jeu réel (captures ci-dessous) : les ennemis suivent la route du début à la fin, y compris dans chaque virage, sans jamais couper à travers l'herbe.

Niveau 5 (double voie) — limite documentée honnêtement (cahier V5, section 3.1 : "si cela est impossible sans dégrader le gameplay, le signaler clairement") : le mécanisme V4 "deux chemins convergents" (deux voies de spawn logiquement distinctes) ne peut pas être recalé sur deux routes visuellement différentes, car la carte fournie ne dessine qu'UNE SEULE porte d'entrée, encadrée de deux tourelles décoratives — jamais deux routes séparées. Fabriquer une seconde route absente de l'asset protégé est interdit ; supprimer le mécanisme de double voie serait une régression gameplay également interdite. Solution retenue : les deux voies logiques démarrent à quelques pixels d'écart, DANS la largeur de la porte unique dessinée, puis rejoignent immédiatement le même tracé — les deux files de spawn et leurs temporisations indépendantes restent pleinement fonctionnelles, mais restent visuellement confondues sur l'unique route dessinée au lieu de sembler sortir de l'herbe à deux endroits différents. Cette limite reste à évaluer visuellement lors de la bêta physique.

1.3 Emplacements constructibles

Chaque emplacement a été repositionné sur une zone herbeuse lisible de la carte, en respectant la garde-fou d'emprise chemin/construction déjà existante depuis la V2 (`findFootprintViolations`, distance minimale requise = 58 unités logiques). Vérification automatisée : **aucune violation détectée sur aucun des 5 niveaux**, avant ET après le recalage. Les emplacements restent des éléments interactifs gérés par le moteur (jamais incrustés dans l'image elle-même).

1.4 Lisibilité et interface

HUD, panneaux, boutons et zones tactiles restent des éléments DOM séparés au-dessus du canvas, inchangés par cette mission. Vérifié sur 5 largeurs mobiles (320/360/390/414/480px) : aucun débordement, aucun scroll de page. Le fond reste fixe et stable.



Vue d'ensemble réelle de la carte Leonardo en jeu (niveau 1) — portail haut, chemin en S, emplacements sur l'herbe, forteresse basse.

2. Intégration du nouveau canon Leonardo

2.1 Vérification de la transparence réelle (cahier V5, section 4)

Constat mesuré AVANT toute intégration : le fichier fourni est un **JPEG RVB**, format qui ne peut structurellement porter aucun canal alpha. Ses pixels de fond forment un **damier gris réellement intégré aux pixels** (alternance mesurée ~237/254, blocs d'environ 16px) — l'artefact classique d'un export à transparence "aplatie" par l'outil source. **Aucune transparence réelle n'existait donc avant traitement**, exactement le cas que le cahier demande de signaler avant toute intégration destructive.

2.2 Reconstruction de la transparence

Une première tentative par simple seuil luminosité+saturation a laissé un résidu de damier visible en jeu (l'une des deux teintes du damier, ~237, produisait une opacité résiduelle d'environ 31% au lieu de 0%) — détecté à l'écran, pas seulement en théorie (voir capture avant/après ci-dessous). Un seuil plus strict aurait, à l'inverse, rongé de vrais reflets clairs du métal poli du canon, qui partagent la même plage de luminosité/saturation que le damier.

Méthode finale retenue : un masque de fond "confiant" strict (luminosité ≥ 230 ET quasi gris pur), puis une **propagation en connexité depuis les seuls bords de l'image** (`scipy.ndimage.label`) — seul le damier est topologiquement relié aux bords à travers ce masque strict ; un reflet clair au milieu du canon, même s'il matche localement le seuil, n'est jamais relié aux bords à travers la silhouette opaque qui l'entoure, donc jamais effacé. Un flou léger de la seule bordure alpha évite un contour en dents de scie. Voir `scripts/process_leonardo_canon.py` pour l'implémentation et le raisonnement complet, y compris l'échec de la première tentative, documenté sans le masquer.

2.3 Intégration technique

Le canon est intégré au même registre d'assets réutilisable que la tour d'archers (`TOWER_SPRITE_CONFIG`, `src/ui/render.js`) : chargement via `loadSprite()`, dimensionnement proportionnel, ancrage bas, repli silencieux sur l'ancienne silhouette Canvas si le chargement échoue (jamais d'écran cassé). Aucune statistique, coût, cadence, dégâts ou portée du Canon n'a été modifié. L'indicateur de palier V4 (anneau creux / disque plein) continue de fonctionner sans altérer l'image elle-même.



Canon Leonardo réellement posé sur un emplacement (niveau 2), transparence propre, aucun damier résiduel.

3. Tests et non-régressions (cahier V5, section 6)

Bloc	Validation obtenue
Héritage V4	77/77 tests unitaires toujours passants, aucune assertion supprimée ni affaiblie.
Carte	Chargement réel (HTTP 200), rendu confirmé non replié sur le fond de secours, aucune déformation (ratio 0,5713 vs cible 0,5714), aucun débordement/scroll sur 5 largeurs mobiles.
Chemin	Ennemis visuellement sur la route du début à la fin, y compris dans chaque virage — vérifié par capture réelle en jeu (ci-dessous) et par la garde-fou d'emprise chemin/construction (0 violation).
Construction	Chaque emplacement recalé reste cliquable, constructible et libéré après revente (0 violation d'emprise sur les 5 niveaux).
Canon	Construction, sélection, amélioration, indicateur de palier, revente et libération d'emplacement vérifiés bout-en-bout via une vraie interaction UI (nouveau test mobile dédié).
Autres tours	Archer et longue portée fonctionnellement inchangés (aucun code de leurs familles modifié).
Sauvegarde	Le correctif V4 de reprise de progression et les cycles de rechargement réels restent passants sans modification.

PWA	Manifest/service worker/déploiement toujours fonctionnels, testés sur dimensions smartphone réelles.
-----	--

Suite complète : **77/77 tests unitaires** (node:test) + **31/31 vérifications mobiles/Playwright** (27 héritées de V0 à V4 + 4 nouvelles ou étendues pour la V5 : chargement réel carte+canon, rendu non replié sur le fond de secours, cycle complet construction/sélection/amélioration/revente sur le canon avec le nouveau sprite).



Vague réelle en cours (niveau 1) : les ennemis avancent visiblement le long de la route dessinée, y compris dans les virages.

4. Ce qui n'a PAS été modifié (cahier V5, section 5)

Conformément à l'exigence explicite du cahier, rien d'autre n'a été touché que ce qui était strictement nécessaire au calage graphique : moteur de combat, équilibrage, économie, prix des tours, taux de revente V4 (toujours 60%), vagues, progression, conditions de victoire/défaite, sauvegarde, audio, pause, navigation, onboarding, logique des améliorations, tour d'archers Leonardo existante, longue portée, ennemis et système PWA restent identiques au code V4. Aucune monétisation, publicité, SDK, achat intégré ou chantier Android/AAB n'a été introduit.

5. Fichiers modifiés / ajoutés

Fichier	Nature
assets/leonardo/carte_terrain_original.jpg	Ajouté — original protégé, jamais modifié directement
assets/leonardo/canon_original.jpg	Ajouté — original protégé, jamais modifié directement
assets/leonardo/trace_road.py	Ajouté — script d'audit du tracé réel de la route (documentation/reproductibilité)
scripts/process_leonardo_map.py	Ajouté — recadrage proportionnel de la carte
scripts/process_leonardo_canon.py	Ajouté — reconstruction de la transparence réelle du canon
public/assets/map/carte_terrain.jpg	Ajouté — carte traitée, servie par le jeu
public/assets/towers/canon.png	Ajouté — canon traité (transparence réelle), servi par le jeu
src/engine/levels.js	Modifié — PATH_MAP calé sur la route réelle (niveaux 1-4), adaptation documentée niveau 5, emplacements recalés
src/ui/render.js	Modifié — carte Leonardo comme fond (avec repli Canvas), canon dans TOWER_SPRITE_CONFIG, forteresse Canvas non redessinée par-dessus la carte
scripts/check-mobile.mjs	Modifié — nouveau test dédié carte+canon (chargement réel, rendu, cycle complet UI)
docs/screenshots/16_carte_ensemble_v5.png, 17_canon_pose_v5.png, 18_ennemis_chemin_v5.png	Ajoutés — captures réelles exigées par le cahier

6. Git, CI/déploiement — vérification réelle

Étape	Résultat vérifié
Commit de départ (V4 rapport)	14e8d04150661c0d484d8f98a716a242f018ce32
Commit final V5 (code)	bb90760dac6b1d3aab043098b63edf7494276f7d
HEAD distant après push	Vérifié via git ls-remote — identique au commit local
Job CI "build"	success — 77 tests unitaires + 31 vérifications mobiles/Playwright exécutés et passants en CI
Job CI "deploy"	success — log brut lu directement : pages_build_version = bb90760dac6b1d3aab043098b63edf7494276f7d (identique au commit)
Identifiant de build re-vérifié localement	v0.1.0+bb90760 (rebuild avec GITHUB_SHA=HEAD réel)
URL publique (lien de bêta téléphone)	https://scuillerjack-oss.github.io/bastion-line-v0/

Le rapport économique officiel BASTION LINE (étude séparée, cahier économique) a été commité et déployé séparément (commit 5dbd6f8c2630e6e292a8c7f4fdc1e12489827dd1) — sa propre CI/déploiement a été vérifiée indépendamment, sans lien fonctionnel avec le code du jeu.

7. Limites connues et points restant à valider sur téléphone physique

- **Niveau 5 (double voie)** : l'adaptation retenue (voir section 1.2) préserve le mécanisme mais reste visuellement une approximation — sa lisibilité réelle à l'œil humain reste à confirmer en bêta physique.
 - **Recadrage de la carte** : le chemin, le portail et la forteresse sont intégralement préservés, mais du décor latéral secondaire (rochers, arbres, un petit campement) sort du cadre sur les côtés — accepté comme un compromis nécessaire pour atteindre le ratio portrait sans déformation, jamais masqué.
 - **Tracé du chemin** : obtenu par mesure programmatique puis vérification visuelle (superposition), mais reste une approximation par segments droits d'une courbe peinte à main levée — une précision pixel-perfect n'est ni requise ni atteignable avec cette méthode.
 - **Canon** : transparence reconstruite et vérifiée propre en jeu (aucun damier résiduel observé dans les captures), mais le rendu final sur un écran de téléphone physique (contraste, luminosité réelle) reste à confirmer par le porteur.
 - **Cohérence graphique carte/canon/tour d'archers** : les trois assets Leonardo proviennent de générations séparées — leur cohérence stylistique perçue à l'usage réel appartient au porteur, jamais à ce rapport.
-

8. Conclusion

Les deux axes du cahier des charges V5 ont été traités comme une seule mission d'intégration graphique contrôlée : la carte Leonardo est devenue le fond visuel réel du jeu (recadrage proportionnel mesuré, chemin recalé par tracé programmatique, emplacements repositionnés sans violation d'emprise), et le canon Leonardo remplace la silhouette Canvas avec une transparence réellement reconstruite après diagnostic honnête de son absence initiale. Le socle fonctionnel V4 (économie, revente, combat, sauvegarde, vagues) n'a subi aucune modification hors de ce qui était strictement nécessaire au calage graphique, et la seule limite structurelle rencontrée (le mécanisme de double voie du niveau 5 face à un asset à entrée unique) est documentée explicitement plutôt que masquée.

Conformément à l'instruction explicite du cahier, BASTION LINE V5 n'est PAS déclarée esthétiquement validée ou gelée. Ce livrable est une candidate à la bêta physique. La validation visuelle finale et le ressenti tactile réel appartiennent exclusivement au porteur, lors de son propre test sur téléphone physique — les retours de la bêta V4/V5 et de futurs assets graphiques pourront alimenter une V6.